
CS60075 (NLP) Assignment 3

Implementing BERT from Scratch for Fake News Classification

Members: Abhinav Nagar (25CS60R71), Abhishek Kumar Shah (25CS60R50)

Spring Semester 2026, IIT Kharagpur

April 15, 2026

Abstract. We implement a complete encoder-only Transformer (BERT) from scratch in PyTorch, preprocess the Fakeddit Reddit dataset for binary fake news classification, fine-tune the `bert-base-uncased` model from Hugging Face on the prepared data, and interpret the fine-tuned model using Layer Integrated Gradients via the Captum library. All experiments were run on Google Colab with an NVIDIA Tesla T4 GPU, PyTorch 2.10, and Transformers 4.41.

1 Task 1: Implementing BERT from Scratch

The objective of this task is to implement all core components of BERT as object-oriented PyTorch classes, without importing any pre-built BERT layers. The model uses the hyperparameters fixed by the assignment: embedding dimension = 512, number of attention heads = 8, and number of encoder layers = 2.

1.1 Overview of Components

The model is composed of five main classes: `MultiHeadSelfAttention`, `FeedForwardLayer`, `TransformerEncoderLayer`, `BertEmbeddings`, `BertPooler`, and the top-level `BertModel`. Table 1 summarises the data flow.

Table 1: BertModel forward pass summary.

#	Component	Output Shape
1	Input token IDs from tokenizer	(B, S)
2	BertEmbeddings: Token + Positional + Segment	(B, S, 512)
3	TransformerEncoderLayer $\times 2$ (MHSA + FFN + LN)	(B, S, 512)
4	BertPooler: Dense(512 $\rightarrow$ 512) + Tanh on [CLS]	(B, 512)
5	Classifier: Dropout + Linear(512 $\rightarrow$ 2)	(B, 2)

1.2 Multi-Head Self-Attention

The `MultiHeadSelfAttention` module splits the $d_{\text{model}} = 512$ dimensional space into $H = 8$ heads, each of dimension $d_k = d_{\text{model}}/H = 64$. Four learned linear projections are used: $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{512 \times 512}$ and $\mathbf{W}_O \in \mathbb{R}^{512 \times 512}$. For each head, scaled dot-product attention is computed as:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}$$

Padding positions are masked by adding -10^9 to their logits before softmax. The H head outputs are concatenated and projected through $\mathbf{W}_O$ to yield the final MHSA output of shape (B, S, 512).

1.3 Feed-Forward Layer

The `FeedForwardLayer` applies a two-layer MLP identically to every token position:

$$\text{FFN}(\mathbf{x}) = \text{GELU}(\mathbf{x}\mathbf{W}_1 + \mathbf{b}_1) \mathbf{W}_2 + \mathbf{b}_2$$

with $\mathbf{W}_1 \in \mathbb{R}^{512 \times 2048}$ and $\mathbf{W}_2 \in \mathbb{R}^{2048 \times 512}$. The inner dimension $2048 = 4 \times d_{\text{model}}$ follows the BERT convention. GELU activation is used rather than ReLU, consistent with Devlin et al. (2019).

1.4 Transformer Encoder Layer

Each `TransformerEncoderLayer` uses the Post-LayerNorm design of the original BERT paper. It contains two sub-layers, each with a residual connection and Layer Normalisation:

$$\begin{aligned} \text{out}_1 &= \text{LayerNorm}(\mathbf{x} + \text{Dropout}(\text{MHSA}(\mathbf{x}))) \\ \text{out}_2 &= \text{LayerNorm}(\text{out}_1 + \text{Dropout}(\text{FFN}(\text{out}_1))) \end{aligned}$$

The layer returns all three intermediate tensors (MHSA output, FFN output, final output) for shape logging.

1.5 BERT Embeddings

The `BertEmbeddings` module sums three learned embedding tables:

- **Token Embeddings** (30522, 512): maps each token ID to a 512-dim vector.
- **Positional Embeddings** (512, 512): absolute learned position encodings (not sinusoidal), following the original BERT design.
- **Segment Embeddings** (2, 512): distinguish sentence A from B. All token type IDs are set to 0 (sentence A) as required by the assignment.

The element-wise sum is passed through LayerNorm and Dropout.

1.6 Pooler and Classifier Head

`BertPooler` takes the [CLS] token hidden state (position 0) and applies $\text{Dense}(512 \rightarrow 512) + \text{Tanh}$ to produce a sentence-level representation. The classifier applies $\text{Dropout} + \text{Linear}(512 \rightarrow 2)$ to generate the final logits.

1.7 Testing and Shape Verification

The model was tested on the following sample sentence of 17 words, which tokenises to 25 tokens (including [CLS] and [SEP]):

“Scientists have discovered a new species of deep-sea fish that can produce its own light using bioluminescence.”

Tokens: [CLS], scientists, have, discovered, a, new, species, of, deep, -, sea, fish, that, can, produce, its, own, light, using, bio, ##lum, ##ines, ##cence, ., [SEP]

All token type IDs were set to 0. Table 2 records the tensor shape at every required stage.

Table 2: Tensor shapes at each stage of BertModel (sequence length = 25).

Stage	Shape	Description
Input IDs	(1, 25)	Batch= 1, seq_len= 25
Token Embeddings	(1, 25, 512)	Token ID $\rightarrow$ 512-dim vector
Positional Embeddings	(1, 25, 512)	Position 0–24 encoding
Segment Embeddings	(1, 25, 512)	All-zero $\rightarrow$ sentence A
Combined Embeddings (input to L1)	(1, 25, 512)	Sum + LayerNorm + Dropout
Encoder L1 — Output of MHSA	(1, 25, 512)	After $\mathbf{W}_O$ (pre-residual)
Encoder L1 — Output of FFN	(1, 25, 512)	After $\mathbf{W}_2$ (pre-residual)
Encoder L1 — TransformerEncoder out	(1, 25, 512)	After both Add & LN
Encoder L2 — Output of MHSA	(1, 25, 512)	After $\mathbf{W}_O$ (pre-residual)
Encoder L2 — Output of FFN	(1, 25, 512)	After $\mathbf{W}_2$ (pre-residual)
Encoder L2 — TransformerEncoder out	(1, 25, 512)	Final encoder hidden states
Pooled [CLS] Output	(1, 512)	Dense + Tanh on [CLS]
Logits (pre-softmax)	(1, 2)	Raw class scores
Output Probabilities (post-softmax)	(1, 2)	[0.4799, 0.5201]

1.8 Parameter Shapes

Table 3 lists every learnable parameter. The total number of trainable parameters is **22,459,906**.

Table 3: All learnable parameters in the custom BertModel.

Parameter Name	Shape	#Params
embeddings.token_embeddings.weight	(30522, 512)	15,627,264
embeddings.position_embeddings.weight	(512, 512)	262,144
embeddings.segment_embeddings.weight	(2, 512)	1,024
embeddings.norm.weight	(512,)	512
embeddings.norm.bias	(512,)	512
encoder_layers.0.attention.W_q.weight	(512, 512)	262,144
encoder_layers.0.attention.W_q.bias	(512,)	512
encoder_layers.0.attention.W_k.weight	(512, 512)	262,144
encoder_layers.0.attention.W_k.bias	(512,)	512
encoder_layers.0.attention.W_v.weight	(512, 512)	262,144
encoder_layers.0.attention.W_v.bias	(512,)	512
encoder_layers.0.attention.W_o.weight	(512, 512)	262,144
encoder_layers.0.attention.W_o.bias	(512,)	512
encoder_layers.0.feed_forward.linear1.weight	(2048, 512)	1,048,576
encoder_layers.0.feed_forward.linear1.bias	(2048,)	2,048
encoder_layers.0.feed_forward.linear2.weight	(512, 2048)	1,048,576
encoder_layers.0.feed_forward.linear2.bias	(512,)	512
encoder_layers.0.norm1.weight / norm1.bias	(512,) each	512 each
encoder_layers.0.norm2.weight / norm2.bias	(512,) each	512 each
encoder_layers.1 (identical structure to layer 0)	—	4,722,944
pooler.dense.weight	(512, 512)	262,144
pooler.dense.bias	(512,)	512
classifier.1.weight	(2, 512)	1,024
classifier.1.bias	(2,)	2
TOTAL TRAINABLE PARAMETERS	—	22,459,906

2 Task 2: Data Preprocessing

This task prepares the Fakeddit dataset [4] for binary fake news classification. We use the post title field (`clean_title`) and the binary label (`2_way_label`: 0 = Not Fake, 1 = Fake).

2.1 Loading and Combining Files

Three TSV files from the `all_samples` folder were loaded with `pandas.read_csv(sep='\\t')` and concatenated. Only the columns `clean_title`, `2_way_label`, and `id` were retained. Labels were cast to `int`.

Table 4: Rows loaded from each Fakeddit TSV file.

File	Rows Loaded
all_train.tsv	878,218
all_validate.tsv	92,444
all_test_public.tsv	92,444
Combined total	1,063,106

2.2 Cleaning and Original Dataset Statistics

Rows with missing `clean_title` or `2_way_label` values were dropped. Posts whose title contained three or fewer characters were also removed. This step eliminated 95,007 rows (8.9% of the combined dataset).

Table 5: Original dataset statistics after cleaning.

Statistic	Value
Total rows before cleaning	1,063,106
Rows removed (null / too short)	95,007
Total rows after cleaning	968,099
Fake posts (label = 1)	484,632 (50.1%)
Real posts (label = 0)	483,467 (49.9%)

2.3 Balancing the Dataset

To eliminate class-imbalance bias, 2,500 posts were randomly sampled from each class using a fixed random seed of 42, yielding a balanced dataset of 5,000 posts, which was subsequently shuffled.

Table 6: Balanced dataset statistics.

Statistic	Value
Total posts in balanced dataset	5,000
Fake posts (label = 1)	2,500 (50.0%)
Real posts (label = 0)	2,500 (50.0%)

2.4 Train-Test Split

The balanced dataset was split into 80% training and 20% test using `sklearn.model_selection.train_test_split` with `stratify=label` and `random_state=42`, guaranteeing equal class proportions in both sets.

Table 7: Distribution of posts across training and test sets.

Split	Total	Fake (= 1)	Real (= 0)	Ratio
Training	4,000	2,000	2,000	50/50
Test	1,000	500	500	50/50
Total	5,000	2,500	2,500	50/50

3 Task 3: Fine-Tuning BERT for Fake News Classification

For this task we use `BertForSequenceClassification` from Hugging Face [6], loaded with the `bert-base-uncased` pre-trained checkpoint (L= 12, H= 768, A= 12, ~110M parameters). The classification head (Dropout + Linear 768 $\rightarrow$ 2) is randomly initialised and all 109,483,778 parameters are fine-tuned end-to-end. This task does not use the custom `BertModel` from Task 1.

3.1 Tokenisation and Truncation Strategy

Post titles were tokenised using the `bert-base-uncased` `WordPiece` tokeniser with `max_len = 64` tokens. Empirical inspection of the Fakeddit corpus confirms that over 97% of `clean_title` values contain fewer than 62 word-piece tokens, so this setting covers nearly all posts without any truncation.

Table 8: Tokenisation and truncation strategy.

Setting	Value
Maximum sequence length	64 tokens (includes [CLS] and [SEP])
Truncation strategy	Right-truncation: tokens beyond position 62 are dropped
Padding strategy	Right-padding with [PAD] (token ID = 0) to reach <code>max_len</code>
Attention mask	1 for real tokens, 0 for [PAD] positions
Token type IDs	All zeros (single-sentence classification)

3.2 Hyperparameters

Table 9: Fine-tuning hyperparameters.

Hyperparameter	Value
Model checkpoint	<code>bert-base-uncased</code>
Total parameters	109,483,778 (all trainable)
Maximum sequence length	64
Batch size	32
Number of epochs	4
Learning rate	2×10^{-5}
Weight decay	0.01 (non-bias, non-LayerNorm params only)
Warmup ratio	0.10
Warmup steps	50
Total training steps	500 (125 batches $\times$ 4 epochs)
Optimizer	AdamW
LR scheduler	Linear warmup then linear decay to 0
Loss function	CrossEntropyLoss
Gradient clipping	<code>max_norm = 1.0</code>

3.3 Training Results

Table 10 shows per-epoch metrics. Training loss decreases steadily across all four epochs. Test accuracy improves from 76.1% in Epoch 1 to 80.1% in Epoch 2, then stabilises around 79%, indicating mild overfitting from Epoch 3 onward. The divergence between training accuracy (92.3%) and test accuracy (78.7%) at Epoch 4 is expected when fine-tuning a 110M-parameter model on only 4,000 training examples and does not indicate an implementation error.

Table 10: Per-epoch training and test set metrics.

Epoch	Train Loss	Train Acc	Test Loss	Test Acc
1/4	0.5693	69.43%	0.5035	76.10%
2/4	0.4011	82.73%	0.4335	80.10%
3/4	0.2962	88.85%	0.4611	79.50%
4/4	0.2158	92.27%	0.5047	78.70%

3.4 Test Set Evaluation Metrics

The final model checkpoint (Epoch 4) was evaluated on the held-out test set of 1,000 posts (500 fake, 500 real).

Table 11: Overall test set performance metrics.

Metric	Value
Accuracy	0.7870 (78.70%)
Precision (weighted average)	0.7878
Recall (weighted average)	0.7870
F1-Score (weighted average)	0.7868

Table 12: Per-class classification report on the test set.

Class	Precision	Recall	F1-Score	Support
Real (label = 0)	0.80	0.76	0.78	500
Fake (label = 1)	0.77	0.81	0.79	500
Macro average	0.79	0.79	0.79	1,000
Weighted average	0.79	0.79	0.79	1,000

Table 13: Confusion matrix on the test set.

	Predicted: Real (0)	Predicted: Fake (1)
Actual: Real (0)	≈ 379 (True Negative)	≈ 121 (False Positive)
Actual: Fake (1)	≈ 95 (False Negative)	≈ 405 (True Positive)

Note: Confusion matrix values are derived from per-class precision, recall, and support. Exact integer counts are shown in the notebook output figure.

4 Task 4: Integrated Gradients for Token Attribution

4.1 Method

Integrated Gradients (IG) is an axiomatic attribution method introduced by Sundararajan et al. [3]. It assigns importance scores to input features by integrating model gradients along a straight-line path from a baseline $\mathbf{x}'$ to the actual input $\mathbf{x}$:

$$\text{IG}_i(\mathbf{x}) = (x_i - x'_i) \int_{\alpha=0}^1 \frac{\partial F(\mathbf{x}' + \alpha(\mathbf{x} - \mathbf{x}'))}{\partial x_i} d\alpha$$

The integral is approximated via a Riemann sum with `n_steps` uniformly spaced interpolation points. The convergence delta $\delta = |\sum_i IG_i(\mathbf{x}) - (F(\mathbf{x}) - F(\mathbf{x}'))|$ measures how closely the Completeness axiom is satisfied; values below 0.05 are considered acceptable.

4.2 Setup

The fine-tuned model from Task 3 was loaded from disk and set to evaluation mode. A custom forward function was defined for Captum [5]. Because `LayerIntegratedGradients` passes the output of the chosen layer (`word_embeddings`) as its first argument, the function accepts an embedding tensor and forwards it via `inputs_embeds`:

```
def forward_fn(embeddings, attention_mask, token_type_ids):
    output = ig_model(
        inputs_embeds = embeddings,
        attention_mask = attention_mask,
        token_type_ids = token_type_ids)
    return output.logits
```

Table 14: Integrated Gradients configuration.

Setting	Value
Captum object	<code>LayerIntegratedGradients</code>
Attribution layer	<code>bert.embeddings.word_embeddings</code>
Baseline	All-zero token IDs $\rightarrow$ zero embedding vectors
Target class	True label: 1 (Fake) for fake, 0 (Real) for real examples
Interpolation steps	300 (retried at 500 if $\delta > 0.05$)
Score aggregation	L2 norm across embedding dimension $\rightarrow$ one scalar per token
Normalisation	Min-max normalisation to $[0, 1]$
Excluded tokens	<code>[CLS]</code> , <code>[SEP]</code> , <code>[PAD]</code>

4.3 Selected Examples

Six examples were selected from the test set: three correctly-classified fake posts and three correctly-classified real posts. Quality criteria: at least 8 words, no repeated-character patterns, and alphabetic-character ratio above 60%.

Table 15: Examples selected for Integrated Gradients analysis.

ID	Label	Post Title (<code>clean_title</code>)
F1	Fake	katy perry shares nude snap as she plans to vote naked
F2	Fake	attacks in israel weigh heavily on lacrosse team
F3	Fake	a scottish council plans to provide free meals to children who need them days a year in a scheme that will be the first of its kind in the uk if approved at a council meeting next tuesday north lanarkshire proposal comes as teachers report seeing more malnourished pupils
R1	Real	new study reveals humanity just one coexist bumper sticker away from world peace
R2	Real	i want you fdr stay and finish the job campaign poster for fdrs presidential campaign
R3	Real	patrick ekeng dinamo bucharest died at the end soy sauce a new level of alpha

4.4 Attribution Results

Table 16: Top-5 attribution tokens for fake examples (target class = 1).

ID	δ	Rank 1	Rank 2	Rank 3	Rank 4	Rank 5
F1	0.01272	nude (0.810)	snap (0.587)	vote (0.496)	shares (0.348)	naked (0.205)
F2	0.02152	lacrosse (1.000)	weigh (0.876)	israel (0.772)	team (0.499)	attacks (0.206)
F3	0.01664	pupils (0.385)	teachers (0.176)	##hire (0.142)	next (0.126)	##shed (0.126)

Table 17: Top-5 attribution tokens for real examples (target class = 0).

ID	δ	Rank 1	Rank 2	Rank 3	Rank 4	Rank 5
R1	0.00446	reveals (0.456)	study (0.433)	humanity (0.367)	peace (0.269)	new (0.225)
R2	0.01573	poster (0.586)	job (0.409)	presidential (0.406)	##dr (0.399)	campaign (0.393)
R3	0.01428	soy (1.000)	dinamo (0.913)	bucharest (0.745)	died (0.634)	end (0.578)

All six convergence deltas are below 0.022, well within the 0.05 acceptable threshold, confirming that $n_steps = 300$ provides accurate attribution estimates.

4.5 Analysis and Discussion

The attribution results are linguistically coherent and reveal what the model has learned to distinguish real posts from fake ones.

Fake Example F1 — Sensationalist celebrity headline

The tokens *nude* (0.810) and *naked* (0.205) dominate, followed by *snap* and *vote*. The model has learned to associate sexually explicit vocabulary combined with political framing (*vote*) with fabricated content, consistent with known patterns of tabloid-style misinformation.

Fake Example F2 — Implausible context juxtaposition

The niche sports term *lacrosse* (1.000) is the highest-attributed token, followed by *weigh* (0.876) and *israel* (0.772). The model flags the implausible juxtaposition of a geopolitical event with an unrelated sports context as a strong indicator of fabricated content.

Fake Example F3 — Long policy-style headline

Despite the title being truncated to 64 tokens, the educational terms *pupils* (0.385) and *teachers* (0.176) score highest. The subword tokens *##hire* and *##shed* are fragments of “North Lanakshire” and “published” respectively, illustrating how WordPiece tokenises rare proper nouns.

Real Example R1 — Research-framing headline

Research-framing vocabulary *reveals* (0.456) and *study* (0.433) receive the highest scores. The model has learned that “study reveals” framing strongly correlates with credible, factual reporting.

Real Example R2 — Historical political artefact

The top tokens *poster* (0.586), *presidential* (0.406), and *campaign* (0.393) collectively describe a historical political item. Consistent attribution across related tokens shows the model attends to the overall semantic field rather than individual keywords.

Real Example R3 — Named entity factual event

Specific proper nouns *dinamo* (0.913) and *bucharest* (0.745), combined with the factual verb *died* (0.634), drive the real classification. The model associates named entities and verifiable events with authentic posts.

General Observations

Across all six examples the model consistently associates *sensationalist, emotive, or contextually implausible vocabulary* with fake news, and *methodological, institutional, or factual vocabulary* with real news. This is a meaningful learned behaviour that is well explained by Integrated Gradients with $n_steps = 300$ (all $\delta < 0.022$).

References

-
- [1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of NAACL-HLT*, 2019.
 - [2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention Is All You Need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
 - [3] M. Sundararajan, A. Taly, and Q. Yan. Axiomatic Attribution for Deep Networks. In *Proceedings of ICML*, 2017. arXiv:1703.01365.
 - [4] K. Nakamura, S. Levy, and W. Y. Wang. r/Fakeddit: A New Multimodal Benchmark Dataset for Fine-grained Fake News Detection. In *Proceedings of LREC*, 2020.
 - [5] N. Kokhlikyan et al. Captum: A unified and generic model interpretability library for PyTorch, 2020. <https://captum.ai>.
 - [6] T. Wolf et al. Transformers: State-of-the-Art Natural Language Processing. In *Proceedings of EMNLP 2020 System Demonstrations*. Hugging Face, 2020.
 - [7] Fine-tuning BERT: A Practical Guide. <https://medium.com/@whyamit101/fine-tuning-bert-a-practical-guide-b5c94efb3d4d>.
 - [8] Complete Guide to Building BERT from Scratch. <https://medium.com/data-and-beyond/complete-guide-to-building-bert-model-from-scratch-3e656222889>